Forward Propagation: How a Neural Network Predicts

Contents

1. Overview 2
2. The network and the observation 3
3. What one node computes 5
4. Layer 1 as one matrix product 6
    4.1 The weight matrix 6
    4.2 The product 7
5. Layers 2 and 3 8
    5.1 Layer 2 9
    5.2 Layer 3: the prediction 10
    5.3 The shapes at a glance 10
6. The whole network in one formula 11
    6.1 Another way to see it: the forward pass for every input 13
7. Summary 15
8. Sources 15
9. Key terms 15

Where this fits: Pipeline map step 8 (Model). See the Course map.



- **Builds on:** Matrix multiplication as composition; MLP notation and parameter count.
- **Leads to:** Loss functions in deep learning; Backpropagation; Recurrent neural network (RNN).

1. Overview

Key point: To predict, a network passes one observation forward, layer by layer. Each layer is one matrix product, one added bias and one **sigmoid** (G-1798), however many nodes it has.

Forward propagation (G-797) is the computation that turns one **observation** (G-1374) (one record, one row of the data table) into a prediction, moving from the input layer through every hidden layer to the output layer. Forward propagation is how a trained network predicts. The same pass is also the first half of every training step:

1. the network predicts (forward propagation);
2. it measures its error;
3. **backpropagation** (G-247), the training algorithm of later Notes, sends that error backwards to update the weights (previewed in section 10 of the **Jacobian** (G-980)).

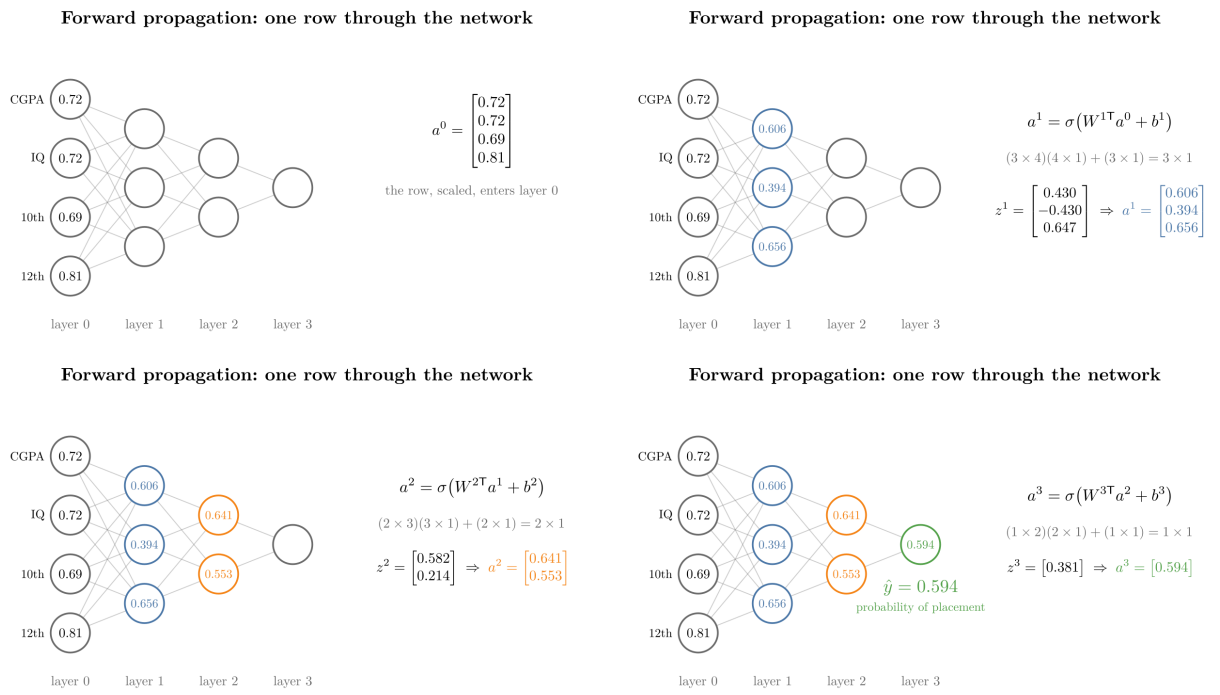


Figure 1: Forward propagation through the 4-3-2-1 network: the values of each layer appear in its nodes, with that layer's matrix equation

Figure 1 shows the whole computation for one student. Each circle is a node (one **perceptron** (G-1486)); the number inside is that node's output for this student; the layers go from left to right, and the equation shown with each layer is the computation that produced its numbers. The rest of this Note works through it, layer by layer, with every number.

2. The network and the observation

> **Key point:** The 4-3-2-1 network has 26 parameters. We feed it one student: CGPA 7.2, IQ 72, 10th marks 69, 12th marks 81.

We use the 4-3-2-1 network and the notation of the MLP notation: 4 inputs, hidden layers of 3 and 2 nodes, and 1 output. The input layer holds the **observation** (G-1374), the two hidden layers are the layers between input and output, and the output layer gives the prediction. Counting the weights and biases, which together are the **parameters** (G-1448) of the network:

$$\text{weights} = 4 \times 3 + 3 \times 2 + 2 \times 1 = 20$$

$$\text{biases} = 3 + 2 + 1 = 6$$

$$\text{total} = 20 + 6 = 26$$

Every node is a **perceptron** (G-1486) with a sigmoid activation. The **sigmoid** (G-1798) is the function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

which turns any number z into a number between 0 and 1 (for example $\sigma(0) = 0.5$), so every node output lies between 0 and 1.

The data has four **features** (G-772) (input variables, one column each) and the **target** (G-1949) *placed* (the output we predict). One observation enters the input layer:

CGPA	IQ	10th marks	12th marks
7.2	72	69	81

Before it enters, we scale the observation into the range 0 to 1: CGPA divided by 10, the others by 100. So the input is $a^0 = (0.72,\ 0.72,\ 0.69,\ 0.81)$.

> **Extra:** Why scale first? With the raw values, the first hidden layer's sums are 20.1, -13.7 and 12.5, and the sigmoid turns them into 1.000, 0.000 and 1.000. Every node is pinned at an extreme. The sigmoid's slope there, $\sigma(z)(1 - \sigma(z))$ (see the sigmoid derivative), is only 1.8×10^{-9}, 1.2×10^{-6} and 3.7×10^{-6}, so a small change to a weight hardly changes the node's output. **Backpropagation** (G-247) multiplies every gradient through a node by this slope, so these weights would barely move in training. The data scaling standardizes the inputs instead (see the **standardization** (G-1874)); dividing by 10 and 100 keeps the numbers here easy to follow.

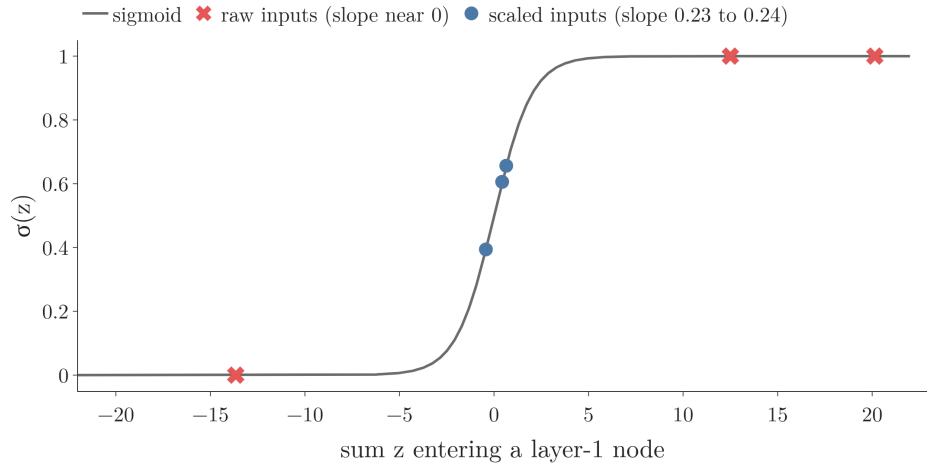


Figure 2: Layer 1's three sums placed on the sigmoid: for the raw student (red crosses) they land on the flat tails, for the scaled student (blue dots) on the steep middle

Figure 2 shows the Extra box as a picture. The horizontal axis is the sum z of a node, and the height is $\sigma(z)$. Watch where the crosses sit. On the flat tails a change in z barely moves $\sigma(z)$, so those nodes cannot learn; the scaled sums sit where the curve is steepest.

The weights are not trained here: we set them by hand to one decimal, so that every step can be checked with a calculator. Training would find better values; **forward propagation** (G-797) works the same either way.

3. What one node computes

> **Key point:** Each node takes the outputs of the previous layer, multiplies each by its weight, adds its bias and applies the **sigmoid** (G-1798).

Every node does what a single perceptron does (see the **perceptron** (G-1486)), with the sigmoid as its **activation** (G-164). For node 1 of layer 1:

1. **In words:** multiply each input by the weight on its connection to this node, add them up, add the node's bias, and pass the total through the sigmoid.
2. **Formula:**

$$z = W_{11}^1 x_1 + W_{21}^1 x_2$$
$$+W_{31}^1 x_3 + W_{41}^1 x_4 + b_{11}$$
$$O_{11} = \sigma(z)$$

3. **Example:** with weights $0.2, 0.4, -0.5, 0.3$ and bias 0.1,

$$0.2 \times 0.72 = 0.144$$
$$0.4 \times 0.72 = 0.288$$
$$-0.5 \times 0.69 = -0.345$$
$$0.3 \times 0.81 = 0.243$$
$$z = 0.144 + 0.288 - 0.345 + 0.243 + 0.1$$
$$z = 0.430$$
$$O_{11} = \sigma(0.430)$$
$$O_{11} = \frac{1}{1 + e^{-0.430}}$$
$$O_{11} = 0.606$$

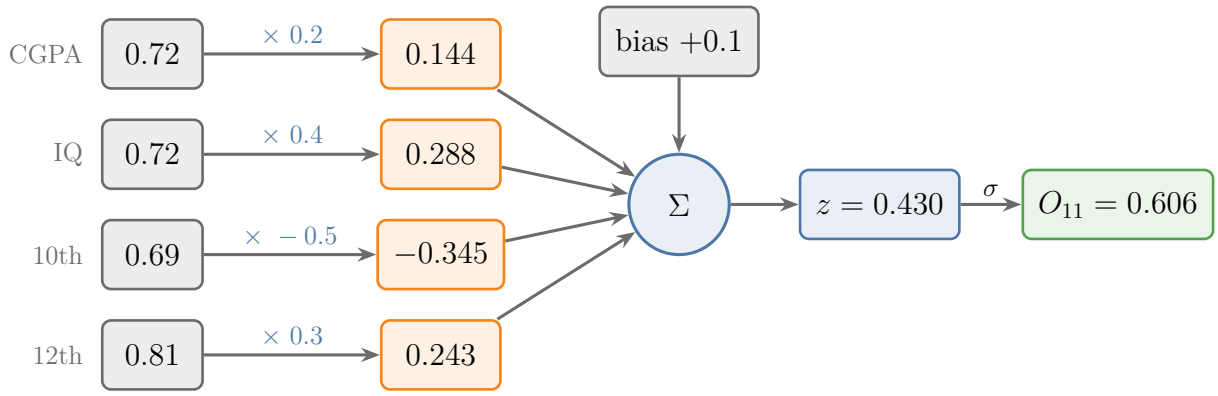


Figure 3: Node 1 of layer 1, step by step: each input times its weight, the four products summed with the bias, then the sigmoid

In Figure 3, follow the numbers left to right: the four products 0.144, 0.288, −0.345 and 0.243 plus the bias 0.1 give $z = 0.430$, and the sigmoid turns it into 0.606.

Doing this node by node works, but a large network has thousands of nodes. Linear algebra does a whole layer in one step.

4. Layer 1 as one matrix product

> **Key point:** Put the layer's weights in a matrix with one row per input node and one column per receiving node; then $z^1 = W^{1\mathsf{T}} a^0 + b^1$ gives all three sums at once.

4.1 The weight matrix

> **Key point:** Row i, column j of W^1 holds W^1_{ij}: from input node i to node j.

We collect the 12 weights entering layer 1 in a 4×3 matrix. Row i holds the weights leaving input node i; column j holds the weights entering node j:

$$W^1 = \begin{bmatrix} W^1_{11} & W^1_{12} & W^1_{13} \\ W^1_{21} & W^1_{22} & W^1_{23} \\ W^1_{31} & W^1_{32} & W^1_{33} \\ W^1_{41} & W^1_{42} & W^1_{43} \end{bmatrix}$$

$$W^1 = \begin{bmatrix} 0.2 & -0.3 & 0.5 \\ 0.4 & 0.1 & -0.2 \\ -0.5 & 0.2 & 0.1 \\ 0.3 & -0.4 & 0.2 \end{bmatrix}$$

$$b^1 = \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix}$$

The first column, $(0.2, 0.4, -0.5, 0.3)$, is exactly the four weights node 1 used in Section 3.

4.2 The product

> **Key point:** Transpose to 3×4, multiply by the 4×1 input, add the 3×1 bias: a 3×1 result, one number per node.

To get one sum per node, each column of W^1 must meet the input. The **transpose** (G-2012) $W^{1\mathsf{T}}$ turns the columns of W^1 into rows (a 4×3 matrix becomes 3×4), and then the product does exactly that (a matrix times a vector, see the **linear transformations** (G-1097), section 7.3).

1. **In words:** multiply the transposed weight matrix by the input vector, add the bias vector, then apply the **sigmoid** (G-1798) to every entry.
2. **Formula:**

$$z^1 = W^{1\mathsf{T}} a^0 + b^1$$

$$a^1 = \sigma(z^1)$$

   The shapes follow the shape rule of the matrix multiplication (section 7.1):

$$(3 \times 4)(4 \times 1) = 3 \times 1$$

$$(3 \times 1) + (3 \times 1) = 3 \times 1$$

3. **Example:**

$$W^{1\mathsf{T}} = \begin{bmatrix} 0.2 & 0.4 & -0.5 & 0.3 \\ -0.3 & 0.1 & 0.2 & -0.4 \\ 0.5 & -0.2 & 0.1 & 0.2 \end{bmatrix}$$

$$a^0 = \begin{bmatrix} 0.72 \\ 0.72 \\ 0.69 \\ 0.81 \end{bmatrix}$$

$$W^{1\mathsf{T}} a^0 = \begin{bmatrix} 0.330 \\ -0.330 \\ 0.447 \end{bmatrix}$$

$$b^1 = \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix}$$

$$z^1 = W^{1\mathsf{T}} a^0 + b^1$$

$$z^1 = \begin{bmatrix} 0.330 \\ -0.330 \\ 0.447 \end{bmatrix} + b^1$$

$$z^1 = \begin{bmatrix} 0.430 \\ -0.430 \\ 0.647 \end{bmatrix}$$

$$a^1 = \sigma(z^1) = \begin{bmatrix} 0.606 \\ 0.394 \\ 0.656 \end{bmatrix}$$

$$a^1 = \begin{bmatrix} O_{11} \\ O_{12} \\ O_{13} \end{bmatrix}$$

each coloured row meets the input and gives
the sum of the node with the same colour

$$\begin{bmatrix} 0.2 & 0.4 & -0.5 & 0.3 \\ -0.3 & 0.1 & 0.2 & -0.4 \\ 0.5 & -0.2 & 0.1 & 0.2 \end{bmatrix} \quad \begin{bmatrix} 0.72 \\ 0.72 \\ 0.69 \\ 0.81 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.430 \\ -0.430 \\ 0.647 \end{bmatrix}$$

$W^{1\mathsf{T}}$: 3×4, one row per node a^0: 4×1 b^1 z^1: 3×1

Figure 4: The product $W^{1\mathsf{T}}a^0 + b^1$ with colours: each row of $W^{1\mathsf{T}}$ (one node's weights) meets the whole input and gives that node's sum

In Figure 4, match the colours: the blue row is the four weights of Figure 3, and it produces the same 0.430. One product computes all three nodes at once.

The first entry, 0.606, is the O_{11} we computed by hand in Section 3. The vector a^1, the outputs of layer 1, is called the **activation** (G-164) of layer 1. The activation a^1 is the input to layer 2.

5. Layers 2 and 3

> **Key point:** The same step repeats: the previous layer's **activation** (G-164) goes in, this layer's activation comes out, until the output layer gives $\hat{y}$.

5.1 Layer 2

> **Key point:** Shapes: 2×3 times 3×1 gives 2×1; the 2×1 bias adds to it.

Layer 2 has 6 weights, from 3 nodes to 2, in a 3×2 matrix:

$$W^2 = \begin{bmatrix} 0.6 & -0.4 \\ -0.2 & 0.5 \\ 0.3 & 0.7 \end{bmatrix}$$

$$b^2 = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$z^2 = W^{2\mathsf{T}} a^1 + b^2$$

$$W^{2\mathsf{T}} = \begin{bmatrix} 0.6 & -0.2 & 0.3 \\ -0.4 & 0.5 & 0.7 \end{bmatrix}$$

$$a^1 = \begin{bmatrix} 0.606 \\ 0.394 \\ 0.656 \end{bmatrix}$$

$$W^{2\mathsf{T}} a^1 = \begin{bmatrix} 0.482 \\ 0.414 \end{bmatrix}$$

$$z^2 = \begin{bmatrix} 0.482 \\ 0.414 \end{bmatrix} + b^2$$

$$z^2 = \begin{bmatrix} 0.582 \\ 0.214 \end{bmatrix}$$

$$a^2 = \sigma(z^2) = \begin{bmatrix} 0.641 \\ 0.553 \end{bmatrix}$$

The two entries are O_{21} and O_{22}.

5.2 Layer 3: the prediction

> **Key point:** Shapes: 1×2 times 2×1 gives 1×1; the 1×1 bias adds to it. A single number, the probability of placement.

The output layer has 2 weights and 1 bias:

$$W^3 = \begin{bmatrix} 0.8 \\ -0.6 \end{bmatrix}$$

$$b^3 = 0.2$$

$$z^3 = W^{3\mathsf{T}} a^2 + b^3$$

$$0.8 \times 0.641 = 0.513$$

$$-0.6 \times 0.553 = -0.332$$

$$z^3 = 0.513 - 0.332 + 0.2$$

$$z^3 = 0.381$$

$$a^3 = \sigma(0.381) = 0.594$$

So $\hat{y} = O_{31} = 0.594$: the network gives this student a 59.4% probability of being placed. Since 0.594 is above 0.5, the network predicts *placed*. Four input numbers became one prediction, using nothing but matrix products, additions and **sigmoids** (G-1798).

5.3 The shapes at a glance

> **Key point:** In every layer the inner sizes of the product match and drop out; what remains is one number per node of that layer.

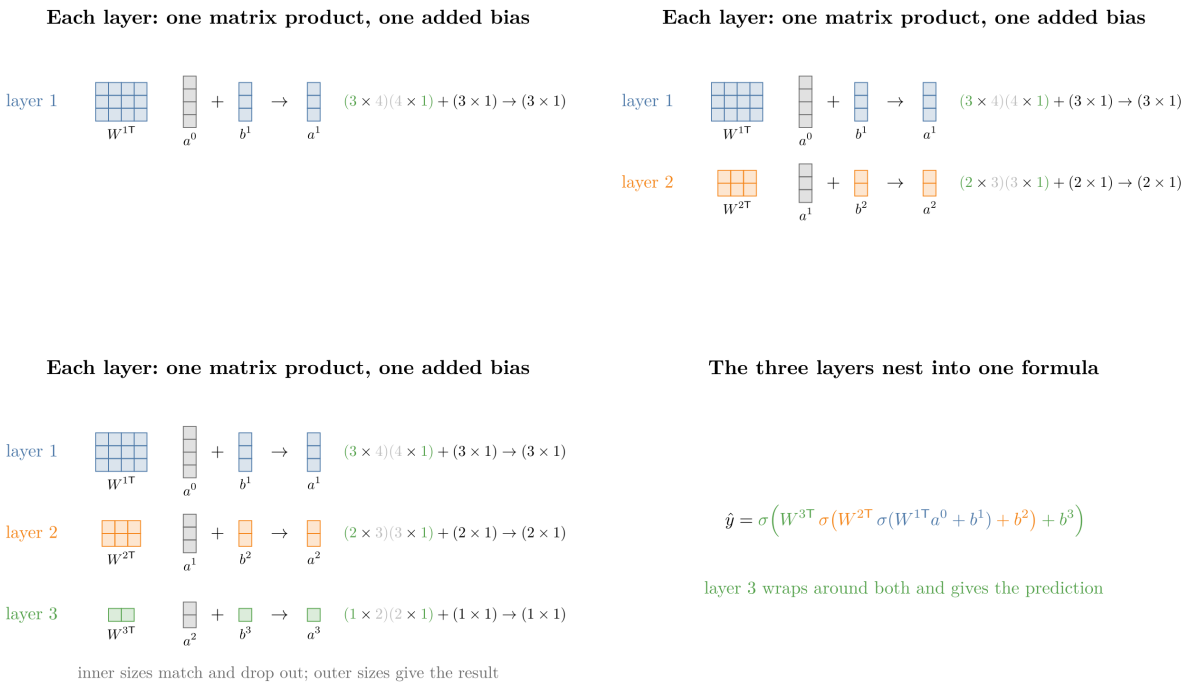


Figure 5: The shapes of the three layers, then the three layers nesting into one formula

Figure 5 lines the three layers up. Watch the two grey numbers in each row: the number of columns of the transposed weight matrix must equal the number of entries of the incoming **activation** (G-164) (4 and 4, then 3 and 3, then 2 and 2). Those inner sizes drop out, and the outer sizes, shown in green, give the shape of the result. The second half of Figure 5 previews section 6: each layer's formula wraps around the one before it.

6. The whole network in one formula

> **Key point:** The whole network is one nested formula: each layer's **sigmoid** (G-1798) wraps around the layer before it. More layers only make the chain longer.

A network works like an assembly line: each station (layer) takes what the previous station handed over, does the same kind of job on it, and passes the result on. Each layer applies the same rule to the **activation** (G-164) of the layer before it:

$$a^k = \sigma\left(W^{k\mathsf{T}}a^{k-1} + b^k\right)$$

for each layer $k = 1, 2, 3$.

Substituting each activation into the next writes the whole network as one nested formula:

$$\hat{y} = a^3 = \sigma\Big(W^{3\mathsf{T}}\ \underbrace{\sigma\Big(W^{2\mathsf{T}}\ \underbrace{\sigma(W^{1\mathsf{T}}a^0 + b^1)}_{a^1} + b^2\Big)}_{a^2} + b^3\Big)$$

The nested formula is what a neural network is, as a function: a chain of matrix products, each followed by a bias and an activation. However large the architecture, prediction stays this organised. The sigmoids between the matrices are essential: without them, the three matrices would collapse into one (see the matrix multiplication, section 7.3), and the network would be no more powerful than one **perceptron** (G-1486).

Python: Forward propagation (G-797) with NumPy.

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

a = np.array([0.72, 0.72, 0.69, 0.81])  # a0
for W, b in [(W1, b1), (W2, b2), (W3, b3)]:
    a = sigmoid(W.T @ a + b)            # one layer
print(a)                                # [0.594]
```

`W1` is the 4×3 array of Section 4.1 and `b1` the bias of length 3; `W.T` is the **transpose** (G-2012) and `@` the matrix product. The same three-line loop works for any number of layers.

Extra: Libraries predict many **observations** (G-1374) at once. Stacking n observations as the rows of an $n \times 4$ matrix X (a 2D tensor, see the **tensors** (G-1957)), the first layer becomes

$$A^1 = \sigma(XW^1 + b^1)$$
$$(n \times 4)(4 \times 3) = n \times 3$$

a matrix with one row of activations per student (for $n = 2$ students, a 2×3 matrix). With rows instead of columns, no transpose is needed (as in section 7.1 of the **linear transformations** (G-1097)). scikit-learn's `MLPClassifier` and Keras both store each layer's weights with shape (nodes in, nodes out), exactly like our W^1, and use this row form: a Keras `Dense` layer computes `activation(dot(input, kernel) + bias)` (Keras docs, `Dense`). The Notebook loads our hand-set weights into both and gets the same 0.594.

6.1 Another way to see it: the forward pass for every input

> **Key point:** Run the forward pass for every possible input and plot the results: each node gives a surface over the inputs, and the output node's surface is the network's prediction everywhere.

So far one student went through the network and one number came out. The same forward pass can be run for every student we can imagine. Figure 6 does this for the two **features** (G-772) CGPA and IQ, each from 0 to 1 after scaling, with the 10th and 12th marks held at this student's 0.69 and 0.81. For each pair (CGPA, IQ) we plot the output of one node as a height, which gives a surface (a sheet over the CGPA-IQ floor).

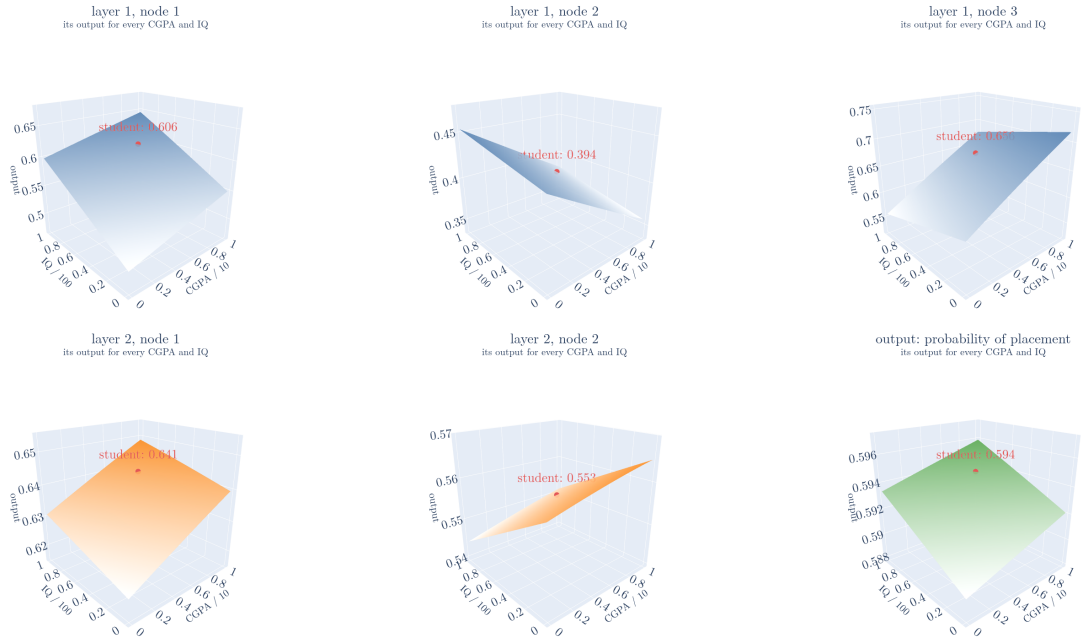


Figure 6: The output of each node of the 4-3-2-1 network for every CGPA and IQ, one node per frame; the red dot is the student of this Note. Each frame has its own height scale. Idea after StatQuest, "Neural Networks Pt. 4: Multiple Inputs and Outputs"

In Figure 6, CGPA and IQ run along the two floor axes and the height is the node's output. Watch the red dot and the heights:

1. **Layer 1.** Each of the three nodes gives a tilted surface. The red dot sits at the heights 0.606, 0.394 and 0.656 of section 4.2.
2. **Layer 2.** Each of the two nodes combines the three surfaces of layer 1 into a new surface. The red dot sits at 0.641 and 0.553.
3. **Output.** The last node combines those two surfaces into the prediction surface, with the red dot at 0.594.

The output surface is the whole network seen as a function: for every input it gives the prediction. With our hand-set weights the surface is almost flat: it only runs from 0.589 to

0.596, so the prediction hardly depends on CGPA or IQ. Training changes the weights, which bends and tilts this surface until it is high for students who are placed and low for those who are not. The MLP intuition shows the surfaces of trained networks.

7. Summary

Layer	Weights	Computation	Product shapes	Activation
1	4×3	$\sigma(W^{1\mathsf{T}}a^0 + b^1)$	$(3 \times 4)(4 \times 1)$	(0.606, 0.394, 0.656)
2	3×2	$\sigma(W^{2\mathsf{T}}a^1 + b^2)$	$(2 \times 3)(3 \times 1)$	(0.641, 0.553)
3	2×1	$\sigma(W^{3\mathsf{T}}a^2 + b^3)$	$(1 \times 2)(2 \times 1)$	$\hat{y} = 0.594$

- Forward propagation: one **observation** (G-1374) moves from the input layer to the output, layer by layer, because each layer needs the outputs of the layer before it.
- Each layer: weighted sums ($W^{\mathsf{T}}a$), plus bias, then **sigmoid** (G-1798), so one matrix product handles every node of a layer, however many nodes it has.
- W^k has one row per node of layer $k-1$ and one column per node of layer k, so column j holds the weights entering node j.
- The whole network is one nested formula, each layer wrapped around the one before it (section 6), so more layers only make the chain longer.
- Backpropagation, next, uses this forward pass to compute the error and update the 26 **parameters** (G-1448).

So a network turns one observation into a prediction by repeating the same step per layer: matrix product, bias, sigmoid, from the inputs to $\hat{y}$.

8. Sources

Built from

- CampusX, "Forward Propagation | How a neural network predicts output?", YouTube, https://www.youtube.com/watch?v=7MuiScUkboE
- StatQuest with Josh Starmer, "Neural Networks Pt. 4: Multiple Inputs and Outputs", YouTube, https://www.youtube.com/watch?v=83LYR-1IcjA (section 6.1: the forward pass as a surface over all inputs)

Other references

- Keras documentation, `keras.layers.Dense`.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Forward propagation (G-797)	Passing one row of inputs through the network, layer by layer, to get the prediction.
Activation (a^k) (G-164)	The vector of outputs of layer k; a^0 is the input row.

Term	Meaning
Bias vector (b^k) (G-286)	A column holding one bias per node of layer k; it is added to the layer's weighted sums before the activation.
Weight matrix (W^k) (G-2109)	All weights entering layer k: one row per node of layer $k-1$, one column per node of layer k.
Weighted input (z^k) (G-2116)	The numbers a layer computes before its activation: each node's weighted sum of the previous layer's outputs plus its bias, $z^k = W^{k\mathsf{T}} a^{k-1} + b^k$.
Sigmoid function (G-1798)	An S-shaped function that squashes any number into the range 0 to 1, $\sigma(z) = 1/(1+e^{-z})$; it turns a score into a probability, as in logistic regression.
Observation (G-1374)	One record of a dataset: one row of the data table.
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.
Jacobian (G-980)	The table (matrix) of all first partial derivatives of a function with several inputs and outputs, one row per output and one column per input, $J_{ij} = \partial f_i / \partial x_j$; it shows how every output changes with every input near a point.
Perceptron (G-1486)	The smallest building block of a neural network, one artificial neuron: it multiplies each input by a weight, adds the results and turns the sum into an output.
Parameter (of a function) (G-1448)	A named setting passed to a function, like `sep=";"`.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Standardization (G-1874)	Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable.

Term	Meaning
Transpose (G-2012)	A matrix or vector with rows and columns swapped; it turns a column vector u into a row u^{T}, so $u^{\mathsf{T}}x$ is the dot product.
Linear transformation (G-1097)	A change of the whole plane by a matrix that keeps grid lines straight and evenly spaced, and the origin fixed; algebraically, $L(\mathbf{v}+\mathbf{w}) = L(\mathbf{v}) + L(\mathbf{w})$ and $L(c\mathbf{v}) = cL(\mathbf{v})$.
Tensor (G-1957)	A container of numbers arranged along one or more axes.